

<Nested loop logic, conditions with dictionaries>

নেস্টেড লুপ এবং ডিকশনারির সাথে শর্ত - বিস্তারিত বাংলা ব্যাখ্যা

১. নেস্টেড লুপ (Nested Loop) কি?

নেস্টেড লুপ মানে হলো **একটি লুপের ভিতরে আরেকটি লুপ**। যখন আমাদের একাধিক স্তরের ডেটা প্রসেস করতে হয়, তখন আমরা নেস্টেড লুপ ব্যবহার করি।

সহজ উদাহরণ: ক্লাসের ছাত্রদের মার্কশিট

```
```python
প্রতিটি ক্লাসে একাধিক ছাত্র, প্রতিটি ছাত্রের একাধিক সাবজেক্টে মার্কস
classrooms = {
 '৬ষ্ঠ': ['রহিম', 'করিম', 'সুমি'],
 '৭ম': ['জাবেদ', 'নাদিয়া', 'শুভ'],
 '৮ম': ['তামিম', 'ফারহানা', 'ইমন']
}
```

```
subjects = ['গণিত', 'বাংলা', 'ইংরেজি', 'বিজ্ঞান']
```

# নেস্টেড লুপ - প্রতিটি ক্লাসের জন্য, প্রতিটি ছাত্রের জন্য

```
for class_name, students in classrooms.items():
```

```
 print(f"\n{class_name} শ্রেণী:")
```

```
 print("-" * 30)
```

```
 for student in students:
```

```
 # প্রতিটি ছাত্রের জন্য সাবজেক্টগুলো দেখানো
```

```
 print(f" {student} এর বিষয়সমূহ:")
```

```
 for subject in subjects:
```

```
 print(f" - {subject}")
```

```
 ...
```

**\*\*আউটপুট:\*\***

```
...
```

৬ষ্ঠ শ্রেণী:

-----

রহিম এর বিষয়সমূহ:

- গণিত

- বাংলা
- ইংরেজি
- বিজ্ঞান

করিম এর বিষয়সমূহ:

- গণিত
- বাংলা
- ইংরেজি
- বিজ্ঞান

সুমি এর বিষয়সমূহ:

- গণিত
- বাংলা
- ইংরেজি
- বিজ্ঞান

...

---

## ২. ডিকশনারির সাথে শর্ত (Conditions with Dictionaries)

### উদাহরণ ১: রেস্টুরেন্টের মেনু এবং অর্ডার প্রক্রিয়াকরণ

```
```python
# রেস্টুরেন্টের মেনু
menu = {
    'বার্গার': {'মূল্য': 250, 'স্টক': 10, 'ক্যাটাগরি': 'ফাস্ট ফুড'},
    'পিৎজা': {'মূল্য': 350, 'স্টক': 5, 'ক্যাটাগরি': 'ফাস্ট ফুড'},
    'ফ্রাইড রাইস': {'মূল্য': 180, 'স্টক': 8, 'ক্যাটাগরি': 'মেইন কোর্স'},
    'চিকেন করি': {'মূল্য': 280, 'স্টক': 3, 'ক্যাটাগরি': 'মেইন কোর্স'},
    'আইসক্রিম': {'মূল্য': 120, 'স্টক': 15, 'ক্যাটাগরি': 'ডেজার্ট'},
    'কেক': {'মূল্য': 150, 'স্টক': 4, 'ক্যাটাগরি': 'ডেজার্ট'}
}

# গ্রাহকের অর্ডার
customer_order = ['বার্গার', 'পিৎজা', 'আইসক্রিম', 'চিকেন করি']

print("অর্ডার প্রক্রিয়াকরণ:")
print("=" * 40)

# নেস্টেড লুপ + শর্ত
for item in customer_order:
    if item in menu: # আইটেমটি মেনুতে আছে কিনা
```

```
details = menu[item]

# একাধিক শর্ত চেক করা
if details['স্টক'] > 0:
    if details['মূল্য'] <= 300:
        status = "✅ উপলব্ধ (সাশ্রয়ী)"
    else:
        status = f"✅ উপলব্ধ (মূল্য: {details['মূল্য']} টাকা)"

# ক্যাটাগরি অনুযায়ী মন্তব্য
if details['ক্যাটাগরি'] == 'ডেজার্ট':
    comment = "🍰 মিষ্টি খাবার"
elif details['ক্যাটাগরি'] == 'ফাস্ট ফুড':
    comment = "🍔 ফাস্ট ফুড"
else:
    comment = "🍝 মূল খাবার"

print(f"{item}: {status} - {comment}")
else:
    print(f"❌ {item}: স্টক শেষ!")
else:
    print(f"❌ {item}: মেনুতে নেই")
...

**আউটপুট:**
...

অর্ডার প্রক্রিয়াকরণ:
=====
বার্গার: ✅ উপলব্ধ (সাশ্রয়ী) - 🍔 ফাস্ট ফুড
পিৎজা: ✅ উপলব্ধ (মূল্য: 350 টাকা) - 🍔 ফাস্ট ফুড
আইসক্রিম: ✅ উপলব্ধ (সাশ্রয়ী) - 🍰 মিষ্টি খাবার
চিকেন করি: ✅ উপলব্ধ (মূল্য: 280 টাকা) - 🍝 মূল খাবার
...

---

### উদাহরণ ২: ছাত্রদের রেজাল্ট প্রক্রিয়াকরণ

```python
ছাত্রদের ডেটা
students = {
```

```
'রহিম': {
 'রোল': 1,
 'মার্কস': {'গণিত': 85, 'বাংলা': 78, 'ইংরেজি': 92, 'বিজ্ঞান': 88},
 'শ্রেণী': 'দশম'
},
'করিম': {
 'রোল': 2,
 'মার্কস': {'গণিত': 65, 'বাংলা': 72, 'ইংরেজি': 58, 'বিজ্ঞান': 70},
 'শ্রেণী': 'দশম'
},
'সুমি': {
 'রোল': 3,
 'মার্কস': {'গণিত': 95, 'বাংলা': 88, 'ইংরেজি': 90, 'বিজ্ঞান': 92},
 'শ্রেণী': 'দশম'
},
'জাবেদ': {
 'রোল': 4,
 'মার্কস': {'গণিত': 45, 'বাংলা': 55, 'ইংরেজি': 48, 'বিজ্ঞান': 52},
 'শ্রেণী': 'দশম'
}
}
```

```
print("ছাত্রদের রেজাল্ট বিশ্লেষণ:")
print("=" * 50)
```

```
নেস্টেড লুপ - প্রতিটি ছাত্রের জন্য
for name, info in students.items():
 print(f"\n 🧑 {name} (রোল: {info['রোল']})")
 print(f" শ্রেণী: {info['শ্রেণী']}")
```

```
marks = info['মার্কস']
total = 0
subject_count = 0
```

```
প্রতিটি সাবজেক্টের জন্য
for subject, mark in marks.items():
 total += mark
 subject_count += 1
```

```
শর্ত অনুযায়ী গ্রেড নির্ধারণ
if mark >= 80:
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
grade = "A+"
status = "☀️ অসাধারণ"
elif mark >= 70:
 grade = "A"
 status = "👍 ভালো"
elif mark >= 60:
 grade = "B"
 status = "📖 মোটামুটি"
elif mark >= 40:
 grade = "C"
 status = "⚠️ দুর্বল"
else:
 grade = "F"
 status = "❌ ফেল"

print(f" {subject}: {mark} (গ্রেড: {grade}) - {status}")

গড় এবং ফাইনাল রেজাল্ট
avg = total / subject_count
print(f" 📊 মোট: {total}, গড়: {avg:.1f}")

কন্ডিশনাল চেক - পাশ/ফেল
if avg >= 60:
 print(" ✅ রেজাল্ট: পাশ")
 if avg >= 80:
 print(" 🏆 বিশেষ সম্মাননা!")
 else:
 print(" ❌ রেজাল্ট: ফেল - উন্নতি প্রয়োজন")

print("-" * 40)
'''

উদাহরণ ৩: শপিং মলের বিভিন্ন দোকানের ডেটা প্রক্রিয়াকরণ

'''python
শপিং মলের ডেটা
shopping_mall = {
 'প্রথম তলা': {
 'দোকান': ['ইলেকট্রনিক্স', 'বুকস', 'কসমেটিক্স'],
 'খোলা': True,
```

```
'আয়': {'ইলেকট্রনিক্স': 50000, 'বুকস': 20000, 'কসমেটিক্স': 35000}
},
'দ্বিতীয় তলা': {
 'দোকান': ['ক্লেথিং', 'শু', 'জুয়েলারী'],
 'খোলা': False,
 'আয়': {'ক্লেথিং': 45000, 'শু': 28000, 'জুয়েলারী': 15000}
},
'তৃতীয় তলা': {
 'দোকান': ['ফুড কোর্ট', 'কফি শপ', 'আইসক্রিম'],
 'খোলা': True,
 'আয়': {'ফুড কোর্ট': 30000, 'কফি শপ': 18000, 'আইসক্রিম': 12000}
}
}
```

```
print("শপিং মল ডেটা বিশ্লেষণ:")
print("=" * 50)
```

```
খোলা দোকান গুলো খুঁজে বের করা
open_stores = []
total_income = 0
```

```
নেস্টেড লুপ - প্রতিটি তলার জন্য
for floor, data in shopping_mall.items():
 print(f"\n 🏠 {floor}")
 print(f" স্ট্যাটাস: {'খোলা' if data['খোলা'] else 'বন্ধ'})
```

```
if data['খোলা']:
 print(" দোকানসমূহ:")
```

```
প্রতিটি দোকানের জন্য
for shop in data['দোকান']:
 income = data['আয়'].get(shop, 0)
 total_income += income
```

```
আয়ের উপর ভিত্তি করে ক্যাটাগরি
if income >= 40000:
 category = " ⭐⭐⭐ (উচ্চ)"
 open_stores.append(shop)
elif income >= 25000:
 category = " ⭐⭐ (মধ্যম)"
 open_stores.append(shop)
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
else:
 category = " 🌟 (নিম্ন)"

 print(f" - {shop}: {income} টাকা {category}")
else:
 print(" 🚫 এই তলা বন্ধ রয়েছে")

বন্ধ থাকার কারন বিশ্লেষণ
closed_shops = data['দোকান']
if len(closed_shops) > 2:
 reason = "অর্থনৈতিক কারণে"
else:
 reason = "সংস্কারের কারণে"
print(f" বন্ধের কারন: {reason}")

print(f"\n 🇧🇩 মোট আয়: {total_income} টাকা")
print(f" 🏪 খোলা দোকানের সংখ্যা: {len(open_stores)}")
print(f"খোলা দোকানের তালিকা: {', '.join(open_stores)}")
` ``
```

---

### উদাহরণ ৪: ক্রিকেট টিমের পারফরম্যান্স বিশ্লেষণ

```
` `` python
ক্রিকেট টিমের ডেটা
team = {
 'তামিম': {
 'রান': [45, 78, 12, 89, 34],
 'উইকেট': [0, 1, 0, 0, 1],
 'পজিশন': 'ওপেনার',
 'ম্যাচ': 5
 },
 'মুশফিক': {
 'রান': [56, 34, 92, 45, 67],
 'উইকেট': [0, 0, 0, 0, 0],
 'পজিশন': 'উইকেটরক্ষক',
 'ম্যাচ': 5
 },
 'শাকিব': {
 'রান': [23, 45, 67, 12, 89],
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
'উইকেট': [2, 1, 3, 0, 2],
'পজিশন': 'অলরাউন্ডার',
'ম্যাচ': 5
},
'মিরাজ': {
 'রান': [12, 8, 15, 4, 23],
 'উইকেট': [1, 2, 1, 3, 2],
 'পজিশন': 'বোলার',
 'ম্যাচ': 5
}
}

print("ক্রিকেট টিম পারফরম্যান্স অ্যানালাইসিস:")
print("=" * 60)

প্রতিটি খেলোয়াড়ের জন্য
for player_name, stats in team.items():
 print(f"\n 🏏 {player_name}")
 print(f" পজিশন: {stats['পজিশন']}")
 print(f" ম্যাচ: {stats['ম্যাচ']}")

 runs = stats['রান']
 wickets = stats['উইকেট']

 # রান বিশ্লেষণ
 total_runs = sum(runs)
 avg_runs = total_runs / len(runs) if runs else 0
 highest = max(runs) if runs else 0
 fifties = sum(1 for r in runs if 50 <= r < 100)
 hundreds = sum(1 for r in runs if r >= 100)

 # উইকেট বিশ্লেষণ
 total_wickets = sum(wickets)
 best_bowling = max(wickets) if wickets else 0

 print(f" 🏏 রান:")
 print(f" মোট: {total_runs}, গড়: {avg_runs:.1f}")
 print(f" সর্বোচ্চ: {highest}")
 print(f" ফিফটি: {fifties}, সেঞ্চুরি: {hundreds}")

 # শর্ত অনুযায়ী পারফরম্যান্স রেটিং
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
if total_runs > 250 and total_wickets > 5:
 rating = " 🏆 অসাধারণ (সেরা খেলোয়াড়)"
elif total_runs > 200 or total_wickets > 8:
 rating = " 🌟 চমৎকার"
elif total_runs > 150 or total_wickets > 5:
 rating = " 👍 ভালো"
elif total_runs > 100:
 rating = " 📖 মোটামুটি"
else:
 rating = " ⚠️ উন্নতি প্রয়োজন"

print(f" রেটিং: {rating}")

পজিশন অনুযায়ী বিশেষ মন্তব্য
if stats['পজিশন'] == 'ওপেনার' and avg_runs > 40:
 print(" 🧐 দারুণ ওপেনিং!")
elif stats['পজিশন'] == 'বোলার' and total_wickets > 8:
 print(" 🎯 অসাধারণ বোলিং!")
elif stats['পজিশন'] == 'অলরাউন্ডার' and total_runs > 150 and total_wickets > 5:
 print(" 🔥 পারফেক্ট অলরাউন্ডার!")

print("-" * 50)

পুরো টিমের সামারি
total_team_runs = sum(sum(stats['রান']) for stats in team.values())
total_team_wickets = sum(sum(stats['উইকেট']) for stats in team.values())

print(f"\n 📊 টিম সামারি:")
print(f" মোট রান: {total_team_runs}")
print(f" মোট উইকেট: {total_team_wickets}")
print(f" গড় রান প্রতি খেলোয়াড়: {total_team_runs/len(team):.1f}")
` ``

উদাহরণ ৫: হোটেল রুম বুকিং সিস্টেম

` ``python
হোটেল রুম ডেটা
hotel = {
 'স্ট্যান্ডার্ড': {
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
'রুম': ['101', '102', '103', '104'],
'প্রতি রাতে মূল্য': 2000,
'বুকিং': {
 '101': {'অতিথি': 'আহমেদ', 'দিন': 3, 'চেকইন': '2024-01-15'},
 '103': {'অতিথি': 'রহমান', 'দিন': 2, 'চেকইন': '2024-01-16'}
},
'ডিলাক্স': {
 'রুম': ['201', '202', '203'],
 'প্রতি রাতে মূল্য': 3500,
 'বুকিং': {
 '201': {'অতিথি': 'কাদের', 'দিন': 5, 'চেকইন': '2024-01-14'},
 '202': {'অতিথি': 'হোসেন', 'দিন': 4, 'চেকইন': '2024-01-17'}
 }
},
'সুইট': {
 'রুম': ['301', '302'],
 'প্রতি রাতে মূল্য': 5000,
 'বুকিং': {
 '302': {'অতিথি': 'চৌধুরী', 'দিন': 7, 'চেকইন': '2024-01-10'}
 }
}
}
```

```
print("হোটেল রুম বুকিং স্ট্যাটাস:")
print("=" * 60)
```

```
available_rooms = []
total_revenue = 0
```

# প্রতিটি ক্যাটাগরির জন্য

```
for category, data in hotel.items():
```

```
 print(f"\n 🏠 {category} ক্যাটাগরি (প্রতি রাতে {data['প্রতি রাতে মূল্য']} টাকা)")
```

```
 print(f" মোট রুম: {len(data['রুম'])}")
```

```
 bookings = data['বুকিং']
```

```
 booked_count = len(bookings)
```

```
 available_count = len(data['রুম']) - booked_count
```

```
 print(f" 📌 বুকিংকৃত: {booked_count}, খালি: {available_count}")
```

```
প্রতিটি রুমের জন্য
for room in data['রুম']:
 if room in bookings:
 booking = bookings[room]
 total_cost = booking['দিন'] * data['প্রতি রাতে মূল্য']
 total_revenue += total_cost

 print(f" 🏠 রুম {room}:")
 print(f" অতিথি: {booking['অতিথি']}")
 print(f" দিন: {booking['দিন']}")
 print(f" চেকইন: {booking['চেকইন']}")
 print(f" মোট খরচ: {total_cost} টাকা")

শর্ত চেক - দীর্ঘ সময়ের বুকিং
if booking['দিন'] >= 5:
 print(" ⭐ দীর্ঘমেয়াদী বুকিং (৫+ দিন)")
 # ডিসকাউন্ট অফার
 discount = total_cost * 0.10
 print(f" 🎉 ১০% ডিসকাউন্ট: {discount} টাকা সাশ্রয়")
else:
 available_rooms.append(f"{category} - {room}")
 print(f" 🟢 রুম {room}: খালি (উপলব্ধ)")

রেভিনিউ সারাংশ
print(f"\n 💰 মোট রেভিনিউ: {total_revenue} টাকা")
print(f" 🏠 উপলব্ধ রুম: {len(available_rooms)} টি")
print(f" উপলব্ধ রুমের তালিকা: {' '.join(available_rooms[:5])}...")
` ``

গুরুত্বপূর্ণ টিপস এবং কৌশল

১. নেস্টেড লুপে ব্রেক এবং কন্টিনিউ ব্যবহার

` `` python
ব্রেক - লুপ থেকে বেরিয়ে যাওয়া
data = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 10, 11, 12]
```

```
]
```

```
for row in data:
 for num in row:
 if num == 7:
 print(f"পেয়েছি: {num}")
 break # শুধু ইনার লুপ থেকে বের হবে
 else:
 continue
 break # আউটার লুপ থেকে বের হবে
...`
```

### ২. ডিকশনারিতে একাধিক শর্ত চেক

```
```python
# চেইনড কন্ডিশন
students = [
    {'নাম': 'রহিম', 'বয়স': 20, 'জিপিএ': 3.8, 'উপস্থিতি': 90},
    {'নাম': 'করিম', 'বয়স': 21, 'জিপিএ': 3.2, 'উপস্থিতি': 75},
    {'নাম': 'সুমি', 'বয়স': 19, 'জিপিএ': 3.9, 'উপস্থিতি': 95}
]

for student in students:
    # একাধিক শর্ত একসাথে
    if (student['জিপিএ'] >= 3.5 and
        student['উপস্থিতি'] >= 85 and
        student['বয়স'] <= 22):
        print(f"✅ {student['নাম']}: মেধাবী ছাত্র")
    elif student['জিপিএ'] < 3.0:
        print(f"⚠️ {student['নাম']}: উন্নতি প্রয়োজন")
    else:
        print(f"📖 {student['নাম']}: ভালো, কিন্তু আরও উন্নতি করতে পারে")
...`
```

৩. কম্প্রিহেনশন ব্যবহার করে কোড সংক্ষিপ্ত করা

```
```python
সাধারণ নেস্টেড লুপ
matrix = []
for i in range(3):
 row = []
 for j in range(3):
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
 row.append(i * j)
 matrix.append(row)

কম্প্রিহেনশন দিয়ে
matrix = [[i * j for j in range(3)] for i in range(3)]

ডিকশনারি কম্প্রিহেনশন
squares = {x: x**2 for x in range(1, 6)}
` ``

সাধারণ ভুল এবং তার সমাধান

ভুল ১: ডিপ নেস্টিং

` `` python
❌ ভুল - অনেক বেশি নেস্টেড
for a in list1:
 for b in list2:
 for c in list3:
 for d in list4:
 # খুব জটিল
 pass

✅ সঠিক - ফাংশনে ভাগ করে নিন
def process_combinations(a, b):
 # লজিক
 pass

for a in list1:
 for b in list2:
 process_combinations(a, b)
` ``

ভুল ২: ডিকশনারি মডিফাই করে লুপ চালানো

` `` python
❌ ভুল
my_dict = {'a': 1, 'b': 2, 'c': 3}
for key in my_dict:
 if key == 'b':
 del my_dict[key] # RuntimeError
```

#  সঠিক

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
keys_to_remove = ['b']
```

```
for key in keys_to_remove:
```

```
 if key in my_dict:
```

```
 del my_dict[key]
```

```
...
```

---

এই উদাহরণগুলো থেকে আপনি দেখতে পাচ্ছেন যে নেস্টেড লুপ এবং ডিকশনারির সাথে শর্ত ব্যবহার করে আমরা কীভাবে জটিল ডেটা প্রসেসিং খুব সহজভাবে করতে পারি। বাংলাদেশের প্রেক্ষাপটে এই উদাহরণগুলো আপনাকে দৈনন্দিন কাজে সাহায্য করবে! 

## <Searching in lists, frequency counters>

# লিস্টে সার্চ করা এবং ফ্রিকোয়েন্সি কাউন্টার - বিস্তারিত বাংলা ব্যাখ্যা

## ১. লিস্টে সার্চ করা (Searching in Lists)

### লিস্টে আইটেম খুঁজে বের করার বিভিন্ন উপায়

```
```python
```

```
# আমাদের একটি দোকানের পণ্যের তালিকা
```

```
products = ['চাল', 'ডাল', 'তেল', 'চিনি', 'লবণ', 'ডাল', 'ময়দা', 'চিনি']
```

১. ইন অপারেটর ব্যবহার (সবচেয়ে সহজ)

```
if 'চিনি' in products:
```

```
    print("  চিনি পাওয়া গেছে!")
```

২. ইনডেক্স ব্যবহার (পজিশন জানতে)

```
try:
```

```
    position = products.index('তেল')
```

```
    print(f"  তেলের পজিশন: {position}")
```

```
except ValueError:
```

```
    print("  তেল পাওয়া যায়নি")
```

৩. কাউন্ট ব্যবহার (কতবার আছে)

মডিউল:৮ :: DSA & Problem Solving – Level 2

```
count_sugar = products.count('চিনি')
print(f"চিনি আছে {count_sugar} বার")
```

৪. লুপ দিয়ে সার্চ (নিজেই বানানো)

```
def search_in_list(items, target):
    positions = []
    for i, item in enumerate(items):
        if item == target:
            positions.append(i)
    return positions

result = search_in_list(products, 'ডাল')
print(f"ডাল পাওয়া গেছে ইনডেক্সে: {result}")
```
```

---

### রিয়েল-লাইফ উদাহরণ ১: মোবাইল ফোন কন্ট্যাক্ট সার্চ

```
```python
# ফোনবুক ডেটা
phonebook = [
    {'নাম': 'রহিম', 'নম্বর': '0171-123456', 'বিভাগ': 'সেলস'},
    {'নাম': 'করিম', 'নম্বর': '0182-789012', 'বিভাগ': 'এইচআর'},
    {'নাম': 'সুমি', 'নম্বর': '0193-345678', 'বিভাগ': 'মার্কেটিং'},
    {'নাম': 'জাবেদ', 'নম্বর': '0164-901234', 'বিভাগ': 'আইটি'},
    {'নাম': 'নাদিয়া', 'নম্বর': '0175-567890', 'বিভাগ': 'সেলস'},
    {'নাম': 'শুভ', 'নম্বর': '0186-123456', 'বিভাগ': 'আইটি'},
    {'নাম': 'ফারহানা', 'নম্বর': '0197-789012', 'বিভাগ': 'এইচআর'}
]
```

```
print(" 📞 ফোনবুক সার্চ সিস্টেম")
print("=" * 50)
```

১. নাম দিয়ে সার্চ

```
def search_by_name(name):
    results = []
    for contact in phonebook:
        if name.lower() in contact['নাম'].lower():
            results.append(contact)
    return results
```

মডিউল:৮ :: DSA & Problem Solving – Level 2

২. বিভাগ দিয়ে সার্চ

```
def search_by_department(dept):  
    return [c for c in phonebook if c['বিভাগ'].lower() == dept.lower()]
```

৩. নাম্বারের অংশ দিয়ে সার্চ

```
def search_by_number_pattern(pattern):  
    return [c for c in phonebook if pattern in c['নম্বর']]
```

ব্যবহার উদাহরণ

```
print("\n🔍 'রহিম' খুঁজছি:")
```

```
for contact in search_by_name('রহিম'):  
    print(f"  নাম: {contact['নাম']], নম্বর: {contact['নম্বর']], বিভাগ: {contact['বিভাগ']]")
```

```
print("\n🔍 আইটি বিভাগের কন্ট্যাক্ট:")
```

```
for contact in search_by_department('আইটি'):  
    print(f"  {contact['নাম']] - {contact['নম্বর']]")
```

```
print("\n🔍 '017' দিয়ে শুরু নাম্বার:")
```

```
for contact in search_by_number_pattern('017'):  
    print(f"  {contact['নাম']] - {contact['নম্বর']]")
```

৪. উন্নত সার্চ (একাধিক ফিল্টার)

```
def advanced_search(name=None, dept=None, number=None):  
    results = phonebook.copy()
```

```
    if name:
```

```
        results = [c for c in results if name.lower() in c['নাম'].lower()]
```

```
    if dept:
```

```
        results = [c for c in results if c['বিভাগ'].lower() == dept.lower()]
```

```
    if number:
```

```
        results = [c for c in results if number in c['নম্বর']]
```

```
    return results
```

```
print("\n🔍 সেলস বিভাগে 'র' আছে এমন নাম:")
```

```
for contact in advanced_search(name='র', dept='সেলস'):
```

```
    print(f"  {contact['নাম']] - {contact['বিভাগ']]")  
    \ \ \
```

রিয়েল-লাইফ উদাহরণ ২: ই-কমার্স পণ্য সার্চ

```
```python
অনলাইন শপের পণ্য তালিকা
products = [
 {'আইডি': 1, 'নাম': 'স্মার্টফোন', 'ক্যাটাগরি': 'ইলেকট্রনিক্স', 'মূল্য': 25000, 'রেটিং': 4.5},
 {'আইডি': 2, 'নাম': 'ল্যাপটপ', 'ক্যাটাগরি': 'ইলেকট্রনিক্স', 'মূল্য': 65000, 'রেটিং': 4.7},
 {'আইডি': 3, 'নাম': 'হেডফোন', 'ক্যাটাগরি': 'ইলেকট্রনিক্স', 'মূল্য': 1500, 'রেটিং': 4.2},
 {'আইডি': 4, 'নাম': 'টি-শার্ট', 'ক্যাটাগরি': 'ফ্যাশন', 'মূল্য': 800, 'রেটিং': 4.0},
 {'আইডি': 5, 'নাম': 'জিন্স', 'ক্যাটাগরি': 'ফ্যাশন', 'মূল্য': 1200, 'রেটিং': 4.3},
 {'আইডি': 6, 'নাম': 'বই', 'ক্যাটাগরি': 'বুকস', 'মূল্য': 350, 'রেটিং': 4.8},
 {'আইডি': 7, 'নাম': 'পাওয়ার ব্যাংক', 'ক্যাটাগরি': 'ইলেকট্রনিক্স', 'মূল্য': 1200, 'রেটিং': 4.1}
]

class ProductSearch:
 def __init__(self, products):
 self.products = products

 # নাম দিয়ে সার্চ
 def search_by_name(self, keyword):
 return [p for p in self.products if keyword.lower() in p['নাম'].lower()]

 # ক্যাটাগরি দিয়ে সার্চ
 def search_by_category(self, category):
 return [p for p in self.products if p['ক্যাটাগরি'].lower() == category.lower()]

 # মূল্যের রেঞ্জ দিয়ে সার্চ
 def search_by_price_range(self, min_price, max_price):
 return [p for p in self.products if min_price <= p['মূল্য'] <= max_price]

 # রেটিং দিয়ে সার্চ
 def search_by_rating(self, min_rating):
 return [p for p in self.products if p['রেটিং'] >= min_rating]

 # কন্সাইন্ড সার্চ (একাধিক শর্ত)
 def advanced_search(self, **filters):
 results = self.products.copy()

 if 'keyword' in filters:
 results = [p for p in results if filters['keyword'].lower() in p['নাম'].lower()]

 if 'category' in filters:
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
results = [p for p in results if p['ক্যাটাগরি'].lower() == filters['category'].lower()]

if 'min_price' in filters:
 results = [p for p in results if p['মূল্য'] >= filters['min_price']]

if 'max_price' in filters:
 results = [p for p in results if p['মূল্য'] <= filters['max_price']]

if 'min_rating' in filters:
 results = [p for p in results if p['রেটিং'] >= filters['min_rating']]

return results

ফলাফল প্রিন্ট করার ফাংশন
def print_results(self, results, title="সার্চ ফলাফল"):
 print(f"\n 📄 {title}:")
 print("-" * 60)
 if not results:
 print("❌ কোন পণ্য পাওয়া যায়নি")
 return

 for p in results:
 print(f" 📦 {p['নাম']} (ID: {p['আইডি']})")
 print(f" ক্যাটাগরি: {p['ক্যাটাগরি']}, মূল্য: {p['মূল্য']} টাকা, রেটিং: {p['রেটিং']} ⭐")
 print(f" মোট {len(results)} টি পণ্য পাওয়া গেছে")

ব্যবহার
search = ProductSearch(products)

বিভিন্ন ধরনের সার্চ
search.print_results(
 search.search_by_name('ফোন'),
 "🔍 'ফোন' কীওয়ার্ডে সার্চ"
)

search.print_results(
 search.search_by_price_range(1000, 5000),
 "💰 ১০০০-৫০০০ টাকার পণ্য"
)

search.print_results(
 search.advanced_search(
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
category='ইলেকট্রনিক্স',
min_price=1000,
max_price=30000,
min_rating=4.0
),
"🔗 ইলেকট্রনিক্স, ১০০০-৩০০০০ টাকা, ৪+ রেটিং"
)
...

```

### ## ২. ফ্রিকোয়েন্সি কাউন্টার (Frequency Counters)

#### ### বেসিক ফ্রিকোয়েন্সি কাউন্টিং

```
```python
# একটি ক্লাসে ছাত্রদের প্রিয় ফল
favorite_fruits = ['আপেল', 'কলা', 'আপেল', 'আম', 'কলা', 'আপেল', 'আম', 'লিচু', 'আম', 'আপেল']

# ১. ম্যানুয়াল কাউন্ট (লুপ দিয়ে)
fruit_count = {}
for fruit in favorite_fruits:
    if fruit in fruit_count:
        fruit_count[fruit] += 1
    else:
        fruit_count[fruit] = 1

print("🍌 প্রিয় ফলের পরিসংখ্যান:")
for fruit, count in fruit_count.items():
    print(f" {fruit}: {count} বার")

# ২. গেট() মেথড ব্যবহার
fruit_count_get = {}
for fruit in favorite_fruits:
    fruit_count_get[fruit] = fruit_count_get.get(fruit, 0) + 1

# ৩. কাউন্টার ক্লাস (Collections থেকে)
from collections import Counter
fruit_counter = Counter(favorite_fruits)
print(f"\nসবচেয়ে জনপ্রিয় ফল: {fruit_counter.most_common(1)[0][0]}
({fruit_counter.most_common(1)[0][1]} বার)")
```
```

---

### রিয়েল-লাইফ উদাহরণ ১: ভোট গণনা সিস্টেম

```
```python
# নির্বাচনের ভোট ডেটা
votes = [
    'আওয়ামী লীগ', 'বিএনপি', 'আওয়ামী লীগ', 'জাপা', 'আওয়ামী লীগ',
    'বিএনপি', 'আওয়ামী লীগ', 'জাপা', 'বিএনপি', 'আওয়ামী লীগ',
    'আওয়ামী লীগ', 'বিএনপি', 'জাপা', 'আওয়ামী লীগ', 'বিএনপি',
    'আওয়ামী লীগ', 'বিএনপি', 'আওয়ামী লীগ', 'আওয়ামী লীগ', 'জাপা',
    'বিএনপি', 'আওয়ামী লীগ', 'আওয়ামী লীগ', 'বিএনপি', 'আওয়ামী লীগ'
]

class VoteCounter:
    def __init__(self, votes):
        self.votes = votes
        self.results = {}
        self.total_votes = len(votes)

    def count_votes(self):
        """ভোট গণনা করা"""
        self.results = {}
        for party in self.votes:
            self.results[party] = self.results.get(party, 0) + 1
        return self.results

    def get_winner(self):
        """বিজয়ী নির্ধারণ"""
        if not self.results:
            self.count_votes()
        max_votes = max(self.results.values())
        winners = [party for party, count in self.results.items() if count == max_votes]
        return winners, max_votes

    def get_percentage(self):
        """প্রতিটি দলের শতাংশ"""
        percentages = {}
        for party, count in self.results.items():
            percentages[party] = (count / self.total_votes) * 100
        return percentages
```

মডিউল:৮ :: DSA & Problem Solving – Level 2

```
def get_ranking(self):
    """ভোট অনুযায়ী র‍্যাঙ্কিং"""
    return sorted(self.results.items(), key=lambda x: x[1], reverse=True)

def print_results(self):
    """ফলাফল প্রিন্ট করা"""
    print("🗳️ নির্বাচনের ফলাফল")
    print("=" * 50)
    print(f"মোট ভোট: {self.total_votes}")
    print()

    # ভোট গণনা
    if not self.results:
        self.count_votes()

    # পার্টি অনুযায়ী ফলাফল
    for party, count in self.get_ranking():
        percentage = (count / self.total_votes) * 100
        # বার চার্ট তৈরি
        bar = '█' * int(percentage / 2)
        print(f"{party:15} | {bar} {count} ({percentage:.1f}%)")

    # বিজয়ী ঘোষণা
    winners, max_votes = self.get_winner()
    print("\n" + "=" * 50)
    if len(winners) == 1:
        print(f"🏆 বিজয়ী: {winners[0]} (পেয়েছে {max_votes} ভোট)")
    else:
        print(f"👯 টাই: {', '.join(winners)} (প্রত্যেকে {max_votes} ভোট পেয়েছে)")

    # মন্তব্য
    if max_votes > (self.total_votes / 2):
        print("✅ নিরঙ্কুশ সংখ্যাগরিষ্ঠতা")
    elif max_votes > (self.total_votes / 3):
        print("🗳️ আপেক্ষিক সংখ্যাগরিষ্ঠতা")
    else:
        print("⚠️ জোট সরকার গঠনের প্রয়োজন")

# ব্যবহার
counter = VoteCounter(votes)
counter.print_results()
...
```

রিয়েল-লাইফ উদাহরণ ২: সোশ্যাল মিডিয়া অ্যানালাইটিক্স

```
```python
সোশ্যাল মিডিয়া পোস্টের ডেটা
posts = [
 {'ইউজার': 'রহিম', 'লাইক': 45, 'শেয়ার': 10, 'কমেন্ট': 5, 'ক্যাটাগরি': 'টেক'},
 {'ইউজার': 'করিম', 'লাইক': 78, 'শেয়ার': 23, 'কমেন্ট': 12, 'ক্যাটাগরি': 'খেলা'},
 {'ইউজার': 'সুমি', 'লাইক': 92, 'শেয়ার': 45, 'কমেন্ট': 28, 'ক্যাটাগরি': 'ভ্রমণ'},
 {'ইউজার': 'জাবেদ', 'লাইক': 34, 'শেয়ার': 8, 'কমেন্ট': 3, 'ক্যাটাগরি': 'টেক'},
 {'ইউজার': 'নাদিয়া', 'লাইক': 67, 'শেয়ার': 19, 'কমেন্ট': 15, 'ক্যাটাগরি': 'খেলা'},
 {'ইউজার': 'রহিম', 'লাইক': 56, 'শেয়ার': 12, 'কমেন্ট': 7, 'ক্যাটাগরি': 'ভ্রমণ'},
 {'ইউজার': 'সুমি', 'লাইক': 88, 'শেয়ার': 34, 'কমেন্ট': 21, 'ক্যাটাগরি': 'টেক'},
 {'ইউজার': 'শুভ', 'লাইক': 23, 'শেয়ার': 5, 'কমেন্ট': 2, 'ক্যাটাগরি': 'খেলা'},
 {'ইউজার': 'নাদিয়া', 'লাইক': 71, 'শেয়ার': 27, 'কমেন্ট': 18, 'ক্যাটাগরি': 'টেক'},
 {'ইউজার': 'করিম', 'লাইক': 45, 'শেয়ার': 15, 'কমেন্ট': 9, 'ক্যাটাগরি': 'ভ্রমণ'},
 {'ইউজার': 'রহিম', 'লাইক': 39, 'শেয়ার': 7, 'কমেন্ট': 4, 'ক্যাটাগরি': 'খেলা'},
 {'ইউজার': 'সুমি', 'লাইক': 95, 'শেয়ার': 51, 'কমেন্ট': 32, 'ক্যাটাগরি': 'খেলা'}
]
```

```
class SocialMediaAnalytics:
 def __init__(self, posts):
 self.posts = posts
 self.total_posts = len(posts)

 def get_user_stats(self):
 """প্রতিটি ইউজারের পরিসংখ্যান"""
 user_stats = {}
 for post in self.posts:
 user = post['ইউজার']
 if user not in user_stats:
 user_stats[user] = {
 'পোস্ট': 0,
 'মোট_লাইক': 0,
 'মোট_শেয়ার': 0,
 'মোট_কমেন্ট': 0,
 'মোট_এনগেজমেন্ট': 0
 }
 user_stats[user]['পোস্ট'] += 1
```

```
 user_stats[user]['মোট_লাইক'] += post['লাইক']
 user_stats[user]['মোট_শেয়ার'] += post['শেয়ার']
 user_stats[user]['মোট_কমেন্ট'] += post['কমেন্ট']
 user_stats[user]['মোট_এনগেজমেন্ট'] += post['লাইক'] + post['শেয়ার'] + post['কমেন্ট']

 return user_stats

def get_category_stats(self):
 """ক্যাটাগরি অনুযায়ী পরিসংখ্যান"""
 cat_stats = {}
 for post in self.posts:
 cat = post['ক্যাটাগরি']
 if cat not in cat_stats:
 cat_stats[cat] = {
 'পোস্ট': 0,
 'মোট_লাইক': 0,
 'মোট_শেয়ার': 0,
 'মোট_এনগেজমেন্ট': 0,
 'গড়_লাইক': 0
 }
 cat_stats[cat]['পোস্ট'] += 1
 cat_stats[cat]['মোট_লাইক'] += post['লাইক']
 cat_stats[cat]['মোট_শেয়ার'] += post['শেয়ার']
 cat_stats[cat]['মোট_এনগেজমেন্ট'] += post['লাইক'] + post['শেয়ার'] + post['কমেন্ট']

 # গড় গণনা
 for cat, stats in cat_stats.items():
 stats['গড়_লাইক'] = stats['মোট_লাইক'] / stats['পোস্ট']

 return cat_stats

def get_top_engagers(self, n=3):
 """শীর্ষ এনগেজমেন্ট পোস্ট"""
 sorted_posts = sorted(self.posts,
 key=lambda x: x['লাইক'] + x['শেয়ার'] + x['কমেন্ট'],
 reverse=True)
 return sorted_posts[:n]

def get_most_active_users(self, n=3):
 """শীর্ষ সক্রিয় ইউজার"""
 user_stats = self.get_user_stats()
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
sorted_users = sorted(user_stats.items(),
 key=lambda x: x[1]['পোস্ট'],
 reverse=True)
return sorted_users[:n]

def print_analysis(self):
 """সম্পূর্ণ বিশ্লেষণ প্রিন্ট"""
 print("🇬🇧 সোশ্যাল মিডিয়া অ্যানালাইটিক্স")
 print("=" * 60)
 print(f"মোট পোস্ট: {self.total_posts}")

 # ইউজার স্ট্যাটস
 print("\n 👤 ইউজার পরিসংখ্যান:")
 print("-" * 40)
 user_stats = self.get_user_stats()
 for user, stats in sorted(user_stats.items(),
 key=lambda x: x[1]['মোট_এনগেজমেন্ট'],
 reverse=True):
 print(f"{user}:")
 print(f" পোস্ট: {stats['পোস্ট']}, লাইক: {stats['মোট_লাইক']}")
 print(f" শেয়ার: {stats['মোট_শেয়ার']}, কমেন্ট: {stats['মোট_কমেন্ট']}")
 print(f" মোট এনগেজমেন্ট: {stats['মোট_এনগেজমেন্ট']}")

 # ক্যাটাগরি স্ট্যাটস
 print("\n 📁 ক্যাটাগরি অনুযায়ী পারফরম্যান্স:")
 print("-" * 40)
 cat_stats = self.get_category_stats()
 for cat, stats in sorted(cat_stats.items(),
 key=lambda x: x[1]['মোট_এনগেজমেন্ট'],
 reverse=True):
 print(f"{cat}:")
 print(f" পোস্ট: {stats['পোস্ট']}, গড় লাইক: {stats['গড়_লাইক']:.1f}")
 print(f" মোট এনগেজমেন্ট: {stats['মোট_এনগেজমেন্ট']}")

 # শীর্ষ পোস্ট
 print("\n 🏆 শীর্ষ পোস্ট:")
 print("-" * 40)
 top_posts = self.get_top_engagers(3)
 for i, post in enumerate(top_posts, 1):
 total = post['লাইক'] + post['শেয়ার'] + post['কমেন্ট']
 print(f"{i}. {post['ইউজার']} - {post['ক্যাটাগরি']}")
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
print(f" এনগেজমেন্ট: {total} (লাইক: {post['লাইক']}, শেয়ার: {post['শেয়ার']}, কमेंট: {post['কमेंট']})")
```

```
সক্রিয় ইউজার
```

```
print("\n 🔥 শীর্ষ সক্রিয় ইউজার:")
```

```
print("-" * 40)
```

```
active = self.get_most_active_users(3)
```

```
for user, stats in active:
```

```
 print(f"{user}: {stats['পোস্ট']} পোস্ট, {stats['মোট_এনগেজমেন্ট']} এনগেজমেন্ট")
```

```
ব্যবহার
```

```
analytics = SocialMediaAnalytics(posts)
```

```
analytics.print_analysis()
```

```
```\n
```

```
---
```

রিয়েল-লাইফ উদাহরণ ৩: শিক্ষার্থীদের পরীক্ষার ফলাফল বিশ্লেষণ

```
```python
```

```
পরীক্ষার ফলাফল ডেটা
```

```
exam_results = [
```

```
 {'রোল': 1, 'নাম': 'রহিম', 'গ্রেড': 'A', 'মার্ক': 85},
 {'রোল': 2, 'নাম': 'করিম', 'গ্রেড': 'B', 'মার্ক': 72},
 {'রোল': 3, 'নাম': 'সুমি', 'গ্রেড': 'A', 'মার্ক': 92},
 {'রোল': 4, 'নাম': 'জাবেদ', 'গ্রেড': 'C', 'মার্ক': 55},
 {'রোল': 5, 'নাম': 'নাদিয়া', 'গ্রেড': 'A', 'মার্ক': 88},
 {'রোল': 6, 'নাম': 'শুভ', 'গ্রেড': 'B', 'মার্ক': 68},
 {'রোল': 7, 'নাম': 'ফারহানা', 'গ্রেড': 'A', 'মার্ক': 95},
 {'রোল': 8, 'নাম': 'ইমন', 'গ্রেড': 'D', 'মার্ক': 40},
 {'রোল': 9, 'নাম': 'তামিম', 'গ্রেড': 'B', 'মার্ক': 76},
 {'রোল': 10, 'নাম': 'রিয়া', 'গ্রেড': 'C', 'মার্ক': 58},
 {'রোল': 11, 'নাম': 'আরিফ', 'গ্রেড': 'F', 'মার্ক': 32},
 {'রোল': 12, 'নাম': 'পূজা', 'গ্রেড': 'B', 'মার্ক': 79}
```

```
]
```

```
class ExamAnalyzer:
```

```
 def __init__(self, results):
```

```
 self.results = results
```

```
 self.total_students = len(results)
```

```
def get_grade_distribution(self):
 """গ্রেড ডিস্ট্রিবিউশন"""
 grade_count = {}
 for student in self.results:
 grade = student['গ্রেড']
 grade_count[grade] = grade_count.get(grade, 0) + 1
 return grade_count

def get_grade_percentage(self):
 """গ্রেডের শতাংশ"""
 grade_count = self.get_grade_distribution()
 return {grade: (count/self.total_students)*100
 for grade, count in grade_count.items()}

def get_class_statistics(self):
 """ক্লাসের সামগ্রিক পরিসংখ্যান"""
 marks = [s['মার্ক'] for s in self.results]
 return {
 'গড়': sum(marks) / len(marks),
 'সর্বোচ্চ': max(marks),
 'সর্বনিম্ন': min(marks),
 'পাস_হার': sum(1 for m in marks if m >= 40) / len(marks) * 100,
 'এ+_হার': sum(1 for m in marks if m >= 80) / len(marks) * 100
 }

def get_top_performers(self, n=5):
 """শীর্ষ পারফর্মার"""
 sorted_students = sorted(self.results, key=lambda x: x['মার্ক'], reverse=True)
 return sorted_students[:n]

def get_grade_wise_details(self):
 """গ্রেড অনুযায়ী বিস্তারিত"""
 grade_details = {}
 for student in self.results:
 grade = student['গ্রেড']
 if grade not in grade_details:
 grade_details[grade] = {
 'ছাত্ররা': [],
 'মোট_মার্ক': 0,
 'গড়_মার্ক': 0
 }
 grade_details[grade]['ছাত্ররা'].append(student['নাম'])
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
grade_details[grade]['মোট_মার্ক'] += student['মার্ক']

গড় মার্ক গণনা
for grade, details in grade_details.items():
 count = len(details['ছাত্ররা'])
 details['গড়_মার্ক'] = details['মোট_মার্ক'] / count

return grade_details

def print_detailed_report(self):
 """বিস্তারিত রিপোর্ট"""
 print("🇬🇧 পরীক্ষার ফলাফল বিশ্লেষণ রিপোর্ট")
 print("=" * 60)
 print(f"মোট ছাত্র: {self.total_students}")

 # গ্রেড ডিস্ট্রিবিউশন
 print("\n📊 গ্রেড ডিস্ট্রিবিউশন:")
 print("-" * 40)
 grade_percent = self.get_grade_percentage()
 for grade in ['A', 'B', 'C', 'D', 'F']:
 if grade in grade_percent:
 count = self.get_grade_distribution()[grade]
 bar = '█' * int(grade_percent[grade] / 2)
 print(f"গ্রেড {grade}: {bar} {count} জন ({grade_percent[grade]:.1f}%)")

 # ক্লাস স্ট্যাটিস্টিকস
 stats = self.get_class_statistics()
 print("\n📈 ক্লাস পরিসংখ্যান:")
 print("-" * 40)
 print(f"গড় মার্ক: {stats['গড়']:.1f}")
 print(f"সর্বোচ্চ মার্ক: {stats['সর্বোচ্চ']}")
 print(f"সর্বনিম্ন মার্ক: {stats['সর্বনিম্ন']}")
 print(f"পাসের হার: {stats['পাস_হার']:.1f}%")
 print(f"এ+ পাওয়ার হার: {stats['এ+_হার']:.1f}%")

 # গ্রেড অনুযায়ী বিস্তারিত
 print("\n📊 গ্রেড অনুযায়ী বিস্তারিত:")
 print("-" * 40)
 grade_details = self.get_grade_wise_details()
 for grade in sorted(grade_details.keys()):
 details = grade_details[grade]
```

```
print(f"গ্রেড {grade}:")
print(f" ছাত্র: {' '.join(details['ছাত্রা'])}")
print(f" গড় মার্ক: {details['গড়_মার্ক']:.1f}")

শীর্ষ পারফর্মার
print("\n 🏆 শীর্ষ ৫ পারফর্মার:")
print("-" * 40)
top = self.get_top_performers(5)
for i, student in enumerate(top, 1):
 print(f"{i}. {student['নাম']} - {student['মার্ক']} (গ্রেড {student['গ্রেড']})")

মন্তব্য
print("\n 💡 মন্তব্য:")
print("-" * 40)
if stats['পাস_হার'] >= 90:
 print("✅ ক্লাসের পারফরম্যান্স অসাধারণ!")
elif stats['পাস_হার'] >= 75:
 print("👍 ভালো পারফরম্যান্স, আরও উন্নতি সম্ভব")
else:
 print("⚠️ অনেক ছাত্রের উন্নতি প্রয়োজন")

if stats['এ+_হার'] >= 30:
 print("🌟 অনেক মেধাবী ছাত্র আছে!")

ব্যবহার
analyzer = ExamAnalyzer(exam_results)
analyzer.print_detailed_report()
'''

রিয়েল-লাইফ উদাহরণ ৪: ওয়ার্ড ফ্রিকোয়েন্সি অ্যানালাইসিস

'''python
একটি বাংলা টেক্সট
text = """
বাংলাদেশ একটি সুন্দর দেশ। বাংলাদেশের রাজধানী ঢাকা।
ঢাকা বাংলাদেশের বৃহত্তম শহর। বাংলাদেশে অনেক নদী আছে।
বাংলাদেশের মানুষ খুবই অতিথিপরায়ণ। ঢাকায় অনেক মানুষ বাস করে।
বাংলাদেশের অর্থনীতি দ্রুত উন্নয়নশীল। বাংলাদেশের সংস্কৃতি সমৃদ্ধ।
বাংলাদেশের জাতীয় ফুল শাপলা। বাংলাদেশের জাতীয় পাখি দোয়েল।

```

"""

```
class TextAnalyzer:
 def __init__(self, text):
 self.text = text
 # বাংলা স্টপওয়ার্ডস
 self.stopwords = {'একটি', 'আছে', 'এই', 'ও', 'এবং', 'যে', 'তা', 'তিনি', 'তার', 'তাদের'}

 def clean_text(self):
 """টেক্সট পরিষ্কার করা"""
 # বিরামচিহ্ন মুছে ফেলা
 import re
 cleaned = re.sub(r'[,;:!?]', ' ', self.text)
 # শব্দে ভাগ করা
 words = cleaned.split()
 # স্টপওয়ার্ড বাদ দেওয়া এবং ছোট হাতের অক্ষরে কনভার্ট
 words = [w.lower() for w in words if w.lower() not in self.stopwords]
 return words

 def get_word_frequency(self):
 """শব্দের ফ্রিকোয়েন্সি"""
 words = self.clean_text()
 freq = {}
 for word in words:
 freq[word] = freq.get(word, 0) + 1
 return freq

 def get_most_common_words(self, n=10):
 """সবচেয়ে বেশি ব্যবহৃত শব্দ"""
 freq = self.get_word_frequency()
 return sorted(freq.items(), key=lambda x: x[1], reverse=True)[:n]

 def get_word_statistics(self):
 """শব্দ পরিসংখ্যান"""
 words = self.clean_text()
 return {
 'মোট_শব্দ': len(words),
 'ইউনিক_শব্দ': len(set(words)),
 'শব্দ_বৈচিত্র্য': len(set(words)) / len(words) if words else 0,
 'সবচেয়ে_লম্বা_শব্দ': max(words, key=len) if words else ""
 }
```

```
def print_analysis(self):
 """বিশ্লেষণ প্রিন্ট"""
 print("📄 টেক্সট অ্যানালাইসিস রিপোর্ট")
 print("=" * 50)

 # মৌলিক পরিসংখ্যান
 stats = self.get_word_statistics()
 print(f"মোট শব্দ: {stats['মোট_শব্দ']}")
 print(f"ইউনিক শব্দ: {stats['ইউনিক_শব্দ']}")
 print(f"শব্দ বৈচিত্র্য: {stats['শব্দ_বৈচিত্র্য']:.2%}")
 print(f"সবচেয়ে লম্বা শব্দ: {stats['সবচেয়ে_লম্বা_শব্দ']}")

 # শব্দ ফ্রিকোয়েন্সি
 print("\n📊 শব্দ ফ্রিকোয়েন্সি টপ ১০:")
 print("-" * 40)
 common = self.get_most_common_words(10)
 max_count = common[0][1] if common else 1

 for word, count in common:
 bar = '█' * int((count / max_count) * 20)
 print(f"{word:12} | {bar} {count}")

 # প্রাসঙ্গিক মন্তব্য
 print("\n💡 বিশ্লেষণ মন্তব্য:")
 print("-" * 40)
 if stats['শব্দ_বৈচিত্র্য'] > 0.7:
 print("✅ টেক্সটের শব্দ বৈচিত্র্য ভালো")
 elif stats['শব্দ_বৈচিত্র্য'] > 0.5:
```

## Python enumerate() - সম্পূর্ণ বাংলা গাইড

## enumerate() কি?

## মডিউল:৮ :: DSA & Problem Solving – Level 2

`enumerate()` হলো পাইথনের একটি বিল্ট-ইন ফাংশন যা কোনো iterable (যেমন লিস্ট, টাপল, স্ট্রিং) এর উপর লুপ চালানোর সময় \*\*ইনডেক্স (index) এবং ভ্যালু (value) একসাথে\*\* পাওয়ার সুবিধা দেয়।

```
```python
# সাধারণ লুপ (শুধু ভ্যালু পাই)
fruits = ['আপেল', 'কলা', 'আম']
for fruit in fruits:
    print(fruit)

# enumerate() ব্যবহার (ইনডেক্স + ভ্যালু)
for index, fruit in enumerate(fruits):
    print(f"ইনডেক্স {index}: {fruit}")
```
```

```
আউটপুট:
...
ইনডেক্স 0: আপেল
ইনডেক্স 1: কলা
ইনডেক্স 2: আম
...

```

## enumerate() এর গুরুত্বপূর্ণ ফিচারসমূহ

### ১. স্টার্ট প্যারামিটার (কাস্টম ইনডেক্স)

```
```python
# ডিফল্ট ইনডেক্স ০ থেকে শুরু হয়
colors = ['লাল', 'নীল', 'সবুজ']
for i, color in enumerate(colors):
    print(f"{i}: {color}")

# কাস্টম স্টার্ট - ১ থেকে শুরু
for i, color in enumerate(colors, start=1):
    print(f"{i}: {color}")

# রিয়েল-লাইফ উদাহরণ: রোল নম্বর
students = ['রহিম', 'করিম', 'সুমি', 'জাবেদ']
for roll, name in enumerate(students, start=1):
    print(f"রোল {roll}: {name}")
```
```

```

****আউটপুট:****

```

১: লাল

২: নীল

৩: সবুজ

রোল ১: রহিম

রোল ২: করিম

রোল ৩: সুমি

রোল ৪: জাবেদ

```

২. enumerate() এর রিটার্ন টাইপ

```python

# enumerate() একটি enumerate অবজেক্ট রিটার্ন করে

fruits = ['আপেল', 'কলা', 'আম']

enum\_obj = enumerate(fruits)

print(type(enum\_obj)) # <class 'enumerate'>

print(list(enum\_obj)) # [(0, 'আপেল'), (1, 'কলা'), (2, 'আম')]

# enumerate অবজেক্টকে লিস্টে কনভার্ট করা

enum\_list = list(enumerate(fruits, start=1))

print(enum\_list) # [(1, 'আপেল'), (2, 'কলা'), (3, 'আম')]

# enumerate অবজেক্টকে ডিকশনারিতে কনভার্ট করা

enum\_dict = dict(enumerate(fruits, start=1))

print(enum\_dict) # {1: 'আপেল', 2: 'কলা', 3: 'আম'}

```

৩. যেকোনো iterable এর সাথে কাজ করে

```python

# ১. স্ট্রিং এর সাথে

word = 'বাংলা'

for i, char in enumerate(word):

```
print(f"অক্ষর {i}: {char}")
```

# ২. টাপলের সাথে

```
coordinates = [(10, 20), (30, 40), (50, 60)]
```

```
for i, (x, y) in enumerate(coordinates):
```

```
 print(f"পয়েন্ট {i}: X={x}, Y={y}")
```

# ৩. সেটের সাথে (অর্ডার অনিশ্চিত)

```
unique_nums = {5, 2, 8, 1}
```

```
for i, num in enumerate(unique_nums):
```

```
 print(f"{i}: {num}")
```

# ৪. রেঞ্জের সাথে

```
for i, num in enumerate(range(5, 10)):
```

```
 print(f"ইনডেক্স {i}: ভ্যালু {num}")
```

```
...
```

---

## enumerate() এর ব্যবহারিক উদাহরণ

### উদাহরণ ১: টুডু লিস্ট ম্যানেজমেন্ট

```
```python
```

```
# টুডু লিস্ট
```

```
todos = [
```

```
    'দুধ কিনতে হবে',
```

```
    'অফিসের রিপোর্ট জমা দিতে হবে',
```

```
    'বাসা পরিষ্কার করতে হবে',
```

```
    'বন্ধুকে কল করতে হবে'
```

```
]
```

```
print(" 📝 আমার টুডু লিস্ট:")
```

```
print("=" * 40)
```

enumerate দিয়ে টাস্ক নম্বর দেখানো

```
for idx, task in enumerate(todos, start=1):
```

```
    print(f"{idx}. {task}")
```

নির্দিষ্ট টাস্ক আপডেট করা

```
def update_task(task_list, task_num, new_task):
```

```
    if 1 <= task_num <= len(task_list):
```

মডিউল:৮ :: DSA & Problem Solving – Level 2

```
task_list[task_num - 1] = new_task
return True
return False
```

টাস্ক আপডেট

```
update_task(todos, 2, 'অফিসের রিপোর্ট পাঠাতে হবে')
```

```
print("\n 📄 আপডেটেড লিস্ট:")
```

```
for idx, task in enumerate(todos, start=1):
    print(f"{idx}. {task}")
```

কমপ্লিটেড টাস্ক মুছে ফেলা

```
def remove_completed(task_list, completed_indices):
```

```
    # উল্টো ক্রমে মুছে ফেলা (ইনডেক্স নষ্ট না করার জন্য)
```

```
    for idx in sorted(completed_indices, reverse=True):
```

```
        if 0 <= idx < len(task_list):
```

```
            removed = task_list.pop(idx)
```

```
            print(f"✅ সম্পন্ন: {removed}")
```

```
completed = [1, 3] # ১ম এবং ৩য় টাস্ক সম্পন্ন
```

```
remove_completed(todos, completed)
```

```
print("\n 📄 বাকি টাস্ক:")
```

```
for idx, task in enumerate(todos, start=1):
```

```
    print(f"{idx}. {task}")
```

```
...
```

```
---
```

উদাহরণ ২: শিক্ষার্থীদের মার্কশিট প্রক্রিয়াকরণ

```
```python
```

```
ছাত্রদের মার্কস
```

```
students_marks = [
```

```
 {'নাম': 'রহিম', 'বাংলা': 85, 'ইংরেজি': 78, 'গণিত': 92},
```

```
 {'নাম': 'করিম', 'বাংলা': 65, 'ইংরেজি': 72, 'গণিত': 58},
```

```
 {'নাম': 'সুমি', 'বাংলা': 95, 'ইংরেজি': 88, 'গণিত': 90},
```

```
 {'নাম': 'জাবেদ', 'বাংলা': 45, 'ইংরেজি': 55, 'গণিত': 48},
```

```
 {'নাম': 'নাদিয়া', 'বাংলা': 82, 'ইংরেজি': 79, 'গণিত': 85}
```

```
]
```

```
print(" 📊 ছাত্রদের মার্কশিট")
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
print("=" * 60)
```

```
enumerate দিয়ে রোল নম্বর সহ দেখানো
```

```
for roll, student in enumerate(students_marks, start=1):
```

```
 total = student['বাংলা'] + student['ইংরেজি'] + student['গণিত']
```

```
 avg = total / 3
```

```
গ্রেড নির্ধারণ
```

```
if avg >= 80:
```

```
 grade = "A+"
```

```
 status = "🌟 অসাধারণ"
```

```
elif avg >= 70:
```

```
 grade = "A"
```

```
 status = "👍 ভালো"
```

```
elif avg >= 60:
```

```
 grade = "B"
```

```
 status = "📖 মোটামুটি"
```

```
elif avg >= 40:
```

```
 grade = "C"
```

```
 status = "⚠️ দুর্বল"
```

```
else:
```

```
 grade = "F"
```

```
 status = "❌ ফেল"
```

```
print(f"রোল {roll}: {student['নাম']}")
```

```
print(f" বাংলা: {student['বাংলা']}, ইংরেজি: {student['ইংরেজি']}, গণিত: {student['গণিত']}")
```

```
print(f" মোট: {total}, গড়: {avg:.1f}, গ্রেড: {grade}{status}")
```

```
print("-" * 40)
```

```
রোল নম্বর দিয়ে ছাত্র খোঁজা
```

```
def find_student_by_roll(roll):
```

```
 if 1 <= roll <= len(students_marks):
```

```
 return students_marks[roll - 1]
```

```
 return None
```

```
টপার খোঁজা
```

```
def get_topper():
```

```
 max_avg = 0
```

```
 topper_index = 0
```

```
for i, student in enumerate(students_marks):
```

```
 total = student['বাংলা'] + student['ইংরেজি'] + student['গণিত']
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
avg = total / 3
if avg > max_avg:
 max_avg = avg
 topper_index = i
```

```
return topper_index + 1, students_marks[topper_index] # রোল নম্বর + ডেটা
```

```
roll, topper = get_topper()
print(f"\n 🏆 টপার: রোল {roll} - {topper['নাম']} (গড়:
{(topper['বাংলা']+topper['ইংরেজি']+topper['গণিত'])/3:.1f})")
` ``
```

---

### ### উদাহরণ ৩: শপিং কার্ট ম্যানেজমেন্ট

```
` `` python
শপিং কার্ট
cart = [
 {'পণ্য': 'ল্যাপটপ', 'মূল্য': 65000, 'পরিমাণ': 1},
 {'পণ্য': 'মাউস', 'মূল্য': 1200, 'পরিমাণ': 2},
 {'পণ্য': 'কিবোর্ড', 'মূল্য': 3500, 'পরিমাণ': 1},
 {'পণ্য': 'হেডফোন', 'মূল্য': 2500, 'পরিমাণ': 1}
]

class ShoppingCart:
 def __init__(self, items):
 self.items = items

 def display_cart(self):
 """কার্টের আইটেম দেখানো"""
 print(" 🛒 শপিং কার্ট")
 print("=" * 60)
 print(f"{'নং':<4} {'পণ্য':<15} {'মূল্য':<10} {'পরিমাণ':<8} {'সাবটোটাল':<10}")
 print("-" * 60)

 total = 0
 for idx, item in enumerate(self.items, start=1):
 subtotal = item['মূল্য'] * item['পরিমাণ']
 total += subtotal
 print(f"{'idx':<4} {item['পণ্য']:<15} {item['মূল্য']:>6} টাকা {item['পরিমাণ']:>4}টি
 {subtotal:>10} টাকা")
```

```
print("-" * 60)
print(f"{'মোট':<40}{total:>10} টাকা")
return total

def update_quantity(self, item_num, new_qty):
 """আইটেমের পরিমাণ আপডেট"""
 if 1 <= item_num <= len(self.items):
 self.items[item_num - 1]['পরিমাণ'] = new_qty
 return True
 return False

def remove_item(self, item_num):
 """আইটেম মুছে ফেলা"""
 if 1 <= item_num <= len(self.items):
 removed = self.items.pop(item_num - 1)
 return removed
 return None

def apply_discount(self, discount_percent=10):
 """ডিসকাউন্ট প্রয়োগ"""
 total = sum(item['মূল্য'] * item['পরিমাণ'] for item in self.items)
 discount = total * (discount_percent / 100)
 return total - discount

ব্যবহার
cart_obj = ShoppingCart(cart)

print("🛒 বর্তমান কার্ট:")
total = cart_obj.display_cart()

আইটেম আপডেট
print("\n✏ আইটেম আপডেট:")
cart_obj.update_quantity(2, 3) # ২য় আইটেমের পরিমাণ ৩ করলাম
cart_obj.display_cart()

আইটেম মুছে ফেলা
print("\n🗑 আইটেম মুছে ফেলা:")
removed = cart_obj.remove_item(3) # ৩য় আইটেম মুছলাম
if removed:
 print(f"মুছে ফেলা হয়েছে: {removed['পণ্য']}")
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
print("\n 🛒 আপডেটেড কার্ট:")
cart_obj.display_cart()
```

# ডিসকাউন্ট

```
print(f"\n 💰 ১০% ডিসকাউন্টের পর মূল্য: {cart_obj.apply_discount(10):.2f} টাকা")
...

```

### উদাহরণ ৪: CSV ফাইল প্রসেসিং (ডেটা অ্যানালাইসিস)

```
```python
```

স্টুডেন্ট ডেটা (CSV ফরম্যাটে)

```
student_data = [
    ['রোল', 'নাম', 'বিভাগ', 'সেমিস্টার', 'সিজিপিএ'],
    ['101', 'রহিম', 'সিএসই', '৬ষ্ঠ', '3.80'],
    ['102', 'করিম', 'ইইই', '৬ষ্ঠ', '3.45'],
    ['103', 'সুমি', 'সিএসই', '৬ষ্ঠ', '3.92'],
    ['104', 'জাবেদ', 'বিবিএ', '৬ষ্ঠ', '3.20'],
    ['105', 'নাদিয়া', 'সিএসই', '৬ষ্ঠ', '3.75'],
    ['106', 'শুভ', 'ইইই', '৬ষ্ঠ', '3.15']
]
```

```
class DataProcessor:
```

```
    def __init__(self, data):
        self.header = data[0]
        self.rows = data[1:]
```

```
    def display_data(self):
```

```
        """ডেটা দেখানো"""
```

```
        print(" 🇬🇧 স্টুডেন্ট ডেটা")
```

```
        print("=" * 70)
```

হেডার প্রিন্ট

```
        for idx, col in enumerate(self.header):
```

```
            print(f"{col:<12}", end="")
```

```
        print()
```

```
        print("-" * 70)
```

ডেটা রো প্রিন্ট

```
        for row_num, row in enumerate(self.rows, start=1):
```

```
            print(f"{row_num:<2}", end=" ") # সিরিয়াল নম্বর
```

মডিউল:৮ :: DSA & Problem Solving – Level 2

```
        for item in row:
            print(f"{item:<12}", end="")
        print()

def find_by_department(self, dept):
    """বিভাগ অনুযায়ী খোঁজ"""
    results = []
    for row in self.rows:
        if row[2] == dept:
            results.append(row)
    return results

def get_top_performers(self, n=3):
    """শীর্ষ পারফর্মার"""
    # সিজিপিএ অনুযায়ী সাজানো
    sorted_rows = sorted(self.rows, key=lambda x: float(x[4]), reverse=True)
    return sorted_rows[:n]

def get_department_statistics(self):
    """বিভাগ ভিত্তিক পরিসংখ্যান"""
    dept_stats = {}
    for row in self.rows:
        dept = row[2]
        cgpa = float(row[4])
        if dept not in dept_stats:
            dept_stats[dept] = {'মোট': 0, 'ছাত্র': 0, 'গড়': 0}
        dept_stats[dept]['মোট'] += cgpa
        dept_stats[dept]['ছাত্র'] += 1

    # গড় গণনা
    for dept in dept_stats:
        dept_stats[dept]['গড়'] = dept_stats[dept]['মোট'] / dept_stats[dept]['ছাত্র']

    return dept_stats

def export_formatted(self):
    """ফরম্যাটেড ডেটা এক্সপোর্ট"""
    print("\n 📄 ফরম্যাটেড ডেটা:")
    print("-" * 50)

    for idx, row in enumerate(self.rows, start=1):
        print(f"{idx}. {row[1]} (রোল: {row[0]})")
```

মডিউল:৮ :: DSA & Problem Solving – Level 2

```
print(f" বিভাগ: {row[2]}, সেমিস্টার: {row[3]}")
print(f" সিজিপিএ: {row[4]}")

# সিজিপিএ অনুযায়ী মন্তব্য
cgpa = float(row[4])
if cgpa >= 3.80:
    comment = " 🌟 অসাধারণ"
elif cgpa >= 3.50:
    comment = " 👍 খুব ভালো"
elif cgpa >= 3.00:
    comment = " 📖 ভালো"
else:
    comment = " ⚠️ উন্নতি প্রয়োজন"
print(f" মন্তব্য: {comment}")
print()

# ব্যবহার
processor = DataProcessor(student_data)
processor.display_data()

print("\n 🎯 সিএসই বিভাগের ছাত্র:")
cse_students = processor.find_by_department('সিএসই')
for student in cse_students:
    print(f" {student[1]} (রোল: {student[0]}) - সিজিপিএ: {student[4]}")

print("\n 🏆 টপ ৩ পারফর্মার:")
top = processor.get_top_performers(3)
for rank, student in enumerate(top, start=1):
    print(f" #{rank}: {student[1]} - সিজিপিএ: {student[4]}")

print("\n 📊 বিভাগ ভিত্তিক পরিসংখ্যান:")
dept_stats = processor.get_department_statistics()
for dept, stats in dept_stats.items():
    print(f" {dept}: গড় সিজিপিএ {stats['গড়']:.2f} ({stats['ছাত্র']} জন)")

processor.export_formatted()
'''

---
```

উদাহরণ ৫: মাল্টি-ডাইমেনশনাল ডেটা প্রসেসিং

```
python
# ম্যাট্রিক্স ডেটা
matrix = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
]

print("📊 ম্যাট্রিক্স প্রসেসিং")
print("=" * 50)

# ম্যাট্রিক্স প্রিন্ট
print("মূল ম্যাট্রিক্স:")
for row_idx, row in enumerate(matrix):
    for col_idx, value in enumerate(row):
        print(f"{{value:3}}", end=" ")
    print()

# ডায়াগোনাল এলিমেন্ট বের করা
print("\n🔍 ডায়াগোনাল এলিমেন্ট:")
diagonal = []
for idx, row in enumerate(matrix):
    diagonal.append(row[idx])
    print(f"({{idx}},{{idx}}) = {{row[idx]}}")

# সারি এবং কলামের যোগফল
print("\n📊 সারি এবং কলামের যোগফল:")
for row_idx, row in enumerate(matrix):
    row_sum = sum(row)
    print(f"সারি {{row_idx}}: {{row_sum}}")

# কলামের যোগফল
for col_idx in range(len(matrix[0])):
    col_sum = 0
    for row_idx in range(len(matrix)):
        col_sum += matrix[row_idx][col_idx]
    print(f"কলাম {{col_idx}}: {{col_sum}}")

# সাব-ম্যাট্রিক্স এক্সট্রাক্ট
def extract_submatrix(mat, start_row, start_col, size):
    """সাব-ম্যাট্রিক্স বের করা"""
```

```
sub = []
for i in range(start_row, start_row + size):
    row = []
    for j in range(start_col, start_col + size):
        row.append(mat[i][j])
    sub.append(row)
return sub

print("\n  ২x২ সাব-ম্যাট্রিক্স (শুরু ১,১):")
sub = extract_submatrix(matrix, 1, 1, 2)
for row in sub:
    print(row)

# ম্যাট্রিক্স ট্রান্সপোজ
print("\n  ট্রান্সপোজড ম্যাট্রিক্স:")
transposed = [[matrix[j][i] for j in range(len(matrix))] for i in range(len(matrix[0]))]
for row in transposed:
    print(row)

# স্পাইরাল প্রিন্টিং (enumerate এর ব্যবহার)
print("\n  স্পাইরাল অর্ডারে প্রিন্ট:")
def print_spiral(mat):
    top, bottom = 0, len(mat) - 1
    left, right = 0, len(mat[0]) - 1

    while top <= bottom and left <= right:
        # উপরের সারি (বাম থেকে ডান)
        for i in range(left, right + 1):
            print(mat[top][i], end=" ")
        top += 1

        # ডানের কলাম (উপর থেকে নিচ)
        for i in range(top, bottom + 1):
            print(mat[i][right], end=" ")
        right -= 1

        if top <= bottom:
            # নিচের সারি (ডান থেকে বাম)
            for i in range(right, left - 1, -1):
                print(mat[bottom][i], end=" ")
            bottom -= 1

        if left <= right:
```

```
# বামের কলাম (নিচ থেকে উপরে)
for i in range(bottom, top - 1, -1):
    print(mat[i][left], end=" ")
    left += 1

print_spiral(matrix)
'''

---
```

উদাহরণ ৬: রিয়েল-টাইম ডেটা মনিটরিং

```
```python
import time
import random

সেন্সর ডেটা সিমুলেশন
class SensorMonitor:
 def __init__(self):
 self.sensor_readings = []
 self.alerts = []

 def add_reading(self, sensor_id, value):
 """সেন্সর রিডিং যোগ করা"""
 reading = {
 'সেন্সর': sensor_id,
 'ভ্যালু': value,
 'সময়': time.strftime('%H:%M:%S')
 }
 self.sensor_readings.append(reading)

 # অ্যালার্ট চেক
 if value > 80:
 self.alerts.append({
 'সেন্সর': sensor_id,
 'ভ্যালু': value,
 'সময়': reading['সময়'],
 'টাইপ': 'উচ্চ'
 })
 elif value < 20:
 self.alerts.append({
 'সেন্সর': sensor_id,
```

```
'ভ্যালু': value,
'সময়': reading['সময়'],
'টাইপ': 'নিম্ন'
})

def show_recent_readings(self, n=5):
 """সাম্প্রতিক রিডিং দেখানো"""
 print(f"\n 📊 সাম্প্রতিক {n}টি রিডিং:")
 print("=" * 50)

 # enumerate দিয়ে রিভার্সে দেখানো
 for idx, reading in enumerate(reversed(self.sensor_readings[-n:]), start=1):
 print(f"{idx}. সেন্সর {reading['সেন্সর']}: {reading['ভ্যালু']} (সময়: {reading['সময়']})")

def show_alerts(self):
 """অ্যালার্ট দেখানো"""
 if not self.alerts:
 print("✅ কোনো অ্যালার্ট নেই")
 return

 print(f"\n 🚨 অ্যালার্ট ({len(self.alerts)}টি):")
 print("=" * 50)
 for idx, alert in enumerate(self.alerts, start=1):
 status = " ⚠️ " if alert['টাইপ'] == 'উচ্চ' else " ❄️ "
 print(f"{idx}. {status} সেন্সর {alert['সেন্সর']}: {alert['ভ্যালু']} ({alert['টাইপ']}) - {alert['সময়']}")

def get_statistics(self):
 """পরিসংখ্যান"""
 if not self.sensor_readings:
 return

 values = [r['ভ্যালু'] for r in self.sensor_readings]
 print(f"\n 📈 পরিসংখ্যান:")
 print("=" * 50)
 print(f"মোট রিডিং: {len(self.sensor_readings)}")
 print(f"সর্বোচ্চ: {max(values)}")
 print(f"সর্বনিম্ন: {min(values)}")
 print(f"গড়: {sum(values)/len(values):.2f}")
 print(f"অ্যালার্ট: {len(self.alerts)}টি")
```

# সিমুলেশন

```
monitor = SensorMonitor()
```

# ১০টি র্যান্ডম রিডিং জেনারেট

```
print("📡 সেন্সর ডেটা সংগ্রহ চলছে...")
```

```
for i in range(10):
```

```
 sensor_id = random.choice(['A', 'B', 'C'])
```

```
 value = random.randint(10, 90)
```

```
 monitor.add_reading(sensor_id, value)
```

```
 time.sleep(0.5) # ০.৫ সেকেন্ড বিরতি
```

```
monitor.show_recent_readings(10)
```

```
monitor.show_alerts()
```

```
monitor.get_statistics()
```

# সেন্সর আইডি অনুযায়ী ফিল্টার

```
print("\n🔍 সেন্সর A এর রিডিং:")
```

```
a_readings = [r for r in monitor.sensor_readings if r['সেন্সর'] == 'A']
```

```
for idx, reading in enumerate(a_readings, start=1):
```

```
 print(f"{idx}. {reading['ভ্যালু']} (সময়: {reading['সময়']})")
```

```
 ...
```

```

```

## enumerate() vs অন্যান্য পদ্ধতি

```
```python
```

১. রেঞ্জ ব্যবহার (পুরনো পদ্ধতি)

```
fruits = ['আপেল', 'কলা', 'আম']
```

```
print("রেঞ্জ পদ্ধতি:")
```

```
for i in range(len(fruits)):
```

```
    print(f"{i}: {fruits[i]}")
```

২. enumerate() (পাইথনিক পদ্ধতি)

```
print("\nenumerate পদ্ধতি:")
```

```
for i, fruit in enumerate(fruits):
```

```
    print(f"{i}: {fruit}")
```

৩. জিপ ব্যবহার (একাধিক লিস্টের জন্য)

```
names = ['রহিম', 'করিম', 'সুমি']
```

```
ages = [25, 30, 22]
```

```
print("\nজিপ পদ্ধতি:")
```

মডিউল:৮ :: DSA & Problem Solving – Level 2

```
for name, age in zip(names, ages):
    print(f"{name}: {age} বছর")

# 8. enumerate + zip (সবচেয়ে শক্তিশালী)
print("\nenumerate + zip:")
for i, (name, age) in enumerate(zip(names, ages), start=1):
    print(f"{i}. {name}: {age} বছর")
\`\`\`
```

enumerate() এর সুবিধাসমূহ

১. কোড ক্লিনার এবং পঠনযোগ্য

```
\`\`\`python
```

❌ পুরনো স্টাইল

```
for i in range(len(my_list)):
    print(i, my_list[i])
```

✅ পাইথনিক স্টাইল

```
for i, item in enumerate(my_list):
    print(i, item)
\`\`\`
```

২. ইন্ডেক্স ট্র্যাকিং সহজ

```
\`\`\`python
```

ইন্ডেক্স এবং ভ্যালু একসাথে

```
tasks = ['টাস্ক ১', 'টাস্ক ২', 'টাস্ক ৩']
for idx, task in enumerate(tasks):
    if idx == 1: # ২য় এলিমেন্ট চেক
        print(f"বিশেষ টাস্ক: {task}")
\`\`\`
```

৩. কাস্টম স্টার্ট ইন্ডেক্স

```
\`\`\`python
```

রোল নম্বর, সিরিয়াল নম্বর ইত্যাদি

```
students = ['রহিম', 'করিম', 'সুমি']
for roll, name in enumerate(students, start=1):
    print(f"রোল {roll}: {name}")
\`\`\`
```

৪. মেমোরি এফিশিয়েন্ট

```
```python
enumerate() জেনারেটর হিসেবে কাজ করে
পুরো লিস্ট তৈরি করে না, একে একে ভ্যালু দেয়
large_list = range(1000000)
enum = enumerate(large_list) # মেমোরি সাশ্রয়ী
```
```

enumerate() এর সাধারণ ভুল এবং সমাধান

ভুল ১: enumerate অবজেক্ট রি-ইউজ করা

```
```python
❌ ভুল
enum = enumerate([1, 2, 3])
print(list(enum)) # [(0, 1), (1, 2), (2, 3)]
print(list(enum)) # [] - খালি!
```

# ✅ সঠিক

```
data = [1, 2, 3]
for i, val in enumerate(data):
 print(i, val)
আবার ব্যবহার করতে চাইলে নতুন enumerate()
for i, val in enumerate(data):
 print(i, val)
```
```

ভুল ২: লিস্ট মডিফাই করা

```
```python
❌ ভুল - লিস্ট মডিফাই করলে ইনডেক্স পরিবর্তন হয়
numbers = [1, 2, 3, 4, 5]
for i, num in enumerate(numbers):
 if num % 2 == 0:
 numbers.pop(i) # ইনডেক্স পরিবর্তন হয়!
```

# ✅ সঠিক - নতুন লিস্ট তৈরি

```
numbers = [num for num in numbers if num % 2 != 0]
```

# অথবা উল্টো ক্রমে

```
numbers = [1, 2, 3, 4, 5]
for i in range(len(numbers) - 1, -1, -1):
 if numbers[i] % 2 == 0:
```

```
 numbers.pop(i)
 ...

enumerate() এর ব্যবহারের সেরা ক্ষেত্রসমূহ

১. ডেটা ভিজুয়ালাইজেশন
```python
# বার চার্ট তৈরি
data = [10, 25, 15, 30, 20]
labels = ['A', 'B', 'C', 'D', 'E']

for i, value in enumerate(data):
    bar = '█' * value
    print(f"{labels[i]}: {bar} {value}")
...

### ২. ফাইল প্রসেসিং
```python
ফাইল থেকে লাইন নম্বর সহ পড়া
with open('file.txt', 'r') as file:
 for line_num, line in enumerate(file, start=1):
 print(f"লাইন {line_num}: {line.strip()}")
...

৩. ডেটাবেস রেকর্ড প্রসেসিং
```python
# ডেটাবেস কোয়েরি রেজাল্ট প্রসেসিং
records = [
    (1, 'রহিম', 'রাজশাহী'),
    (2, 'করিম', 'ঢাকা'),
    (3, 'সুমি', 'চট্টগ্রাম')
]

for idx, (id, name, city) in enumerate(records, start=1):
    print(f"{idx}. ID: {id}, নাম: {name}, শহর: {city}")
...

### ৪. API রেসপন্স প্রসেসিং
```python
JSON রেসপন্স থেকে ডেটা এক্সট্রাক্ট
```

## মডিউল:৮ :: DSA & Problem Solving – Level 2

```
api_response = {
 'users': [
 {'id': 1, 'name': 'রহিম'},
 {'id': 2, 'name': 'করিম'}
]
}

for idx, user in enumerate(api_response['users'], start=1):
 print(f"ইউজার {idx}: {user['name']} (ID: {user['id']})")
 ...
```

---

### ## সংক্ষিপ্ত সারাংশ

ফিচার	বিবরণ	উদাহরণ
**\*\*বেসিক সিনট্যাক্স\*\***	ইনডেক্স + ভ্যালু	`for i, item in enumerate(list)``
**\*\*স্টার্ট প্যারামিটার\*\***	কাস্টম শুরু ইনডেক্স	`enumerate(list, start=1)``
**\*\*রিটার্ন টাইপ\*\***	enumerate অবজেক্ট	`<class 'enumerate'>``
**\*\*কনভার্সন\*\***	লিস্ট/ডিকশনারিতে	`list(enumerate(data))``
**\*\*মেমোরি\*\***	জেনারেটর হিসেবে	মেমোরি সাশ্রয়ী
**\*\*ইটারেবল\*\***	যেকোনো iterable	স্ট্রিং, লিস্ট, টাপল, সেট

---

**\*\*enumerate()\*\*** পাইথনের একটি অত্যন্ত গুরুত্বপূর্ণ ফাংশন যা কোডকে **\*\*ক্লিনার, রিডেবল এবং পাইথনিক\*\*** করে তোলে। বিশেষ করে যখন আপনার লুপে ইনডেক্স এবং ভ্যালু উভয়ের প্রয়োজন হয়, তখন `enumerate()` ব্যবহার করা উচিত। 🚀